Industry Systems



COMS4113: Fundamentals of Distributed Systems

Instructor: Hubertus Franke

Columbia University

https://frankeh.github.io/cu-ds1-2026-fall

Spanner

Google's geo-distributed, fault-tolerant, transactional database

Outline

- **Why Spanner** - what it is, how Google Ads uses it, and the problem at scale
- **Splits** - how the data is divided, moved and kept together
- **Architecture** - how 2PL, WAL, Paxos and 2PC fit together
- **The performance problem** - locks held across geo-distributed 2PC
- **Lockless reads** via multi-version concurrency control - which needs a global clock
- **TrueTime**: what it guarantees, how it is built, and what it costs
- **Serializable transactions**: which timestamp, and when to pick it
- A worked example, with four cases
- **Spanner in 2026** - and the other bets the industry made on time

What Is Spanner?

Google's globally distributed SQL database. To an application it looks like *one* big relational database; underneath it is thousands of machines in datacenters around the world.

Sharded

Rows are divided into **splits** spread over many servers, and **resharded automatically** as data and servers change.

Replicated

Every split is copied to several replicas in **independent failure zones**, kept in step by **Paxos**. Clients **fail over automatically**.

Transactional

General **ACID transactions** across rows, tables and datacenters - **externally consistent**.

SQL

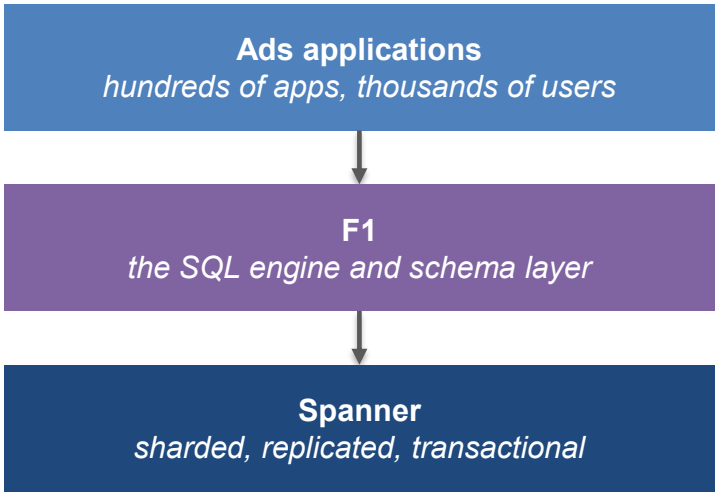
Tables with a **schema** and a SQL query language: **GoogleSQL**, and a **PostgreSQL** dialect.

Designed to scale to *millions of machines*, *hundreds of datacenters* and *trillions of rows* - and, its authors say, the first system to offer externally consistent transactions at global scale.

[Corbett-2012] abstract and section 1.

Spanner Runs Google Ads

How an advertiser's data reaches Spanner



Five replicas: 2 on the US west coast, 3 on the east coast.

100+ TB

of advertising data in one database

100,000s

of requests per second, at peak

10s of trillions

of rows scanned by SQL, every day

99.999%

available - 'five nines' - despite outages

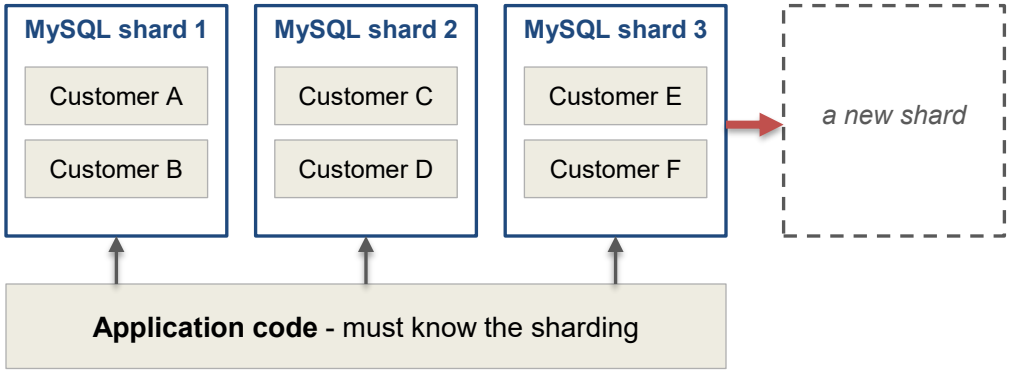
Why a transactional database, not a NoSQL store?

"F1 requires strong transactional semantics, which made using other NoSQL systems impractical." - [Corbett-2012] 5.4

Called AdWords in these papers; renamed Google Ads in July 2018. Numbers: [Shute-2013] section 1 (2013).

The Problem at Scale

Before Spanner: the ads database was **MySQL, sharded by hand.** Each customer, with all of their data, lived on **one fixed shard.**



Resharding took over two years of intense effort, coordinated and tested across **dozens of teams**.

What it bought

Per-customer indexes and queries: a customer's data is on one shard.

What it cost

The application **knows the sharding**, and growth means **resharding by hand**.

The workaround

Data parked in **external Bigtables**: **no transactions**, no queries across all the data.

Tens of terabytes: small next to many NoSQL stores, yet enough to cause difficulties for sharded MySQL. [Corbett-2012] 5.4

What the Problem Demanded

What hurt	What was needed	Spanner's answer
Resharding by hand took over two years and dozens of teams	Grow **without people in the loop**	**Automatic resharding** and migration: splits
MySQL master-slave failover was difficult and risked data loss and downtime	Survive a datacenter outage **without losing writes**	**Synchronous Paxos replication** and automatic failover
Other NoSQL systems could not give transactions across arbitrary data and consistent reads	**Transactions and consistent reads** over any data	General **ACID transactions**, externally consistent

The hard part is having all of it at once

Historically a choice: **NoSQL** stores scale out but give up general transactions; a **relational** database has them but is hard to scale out. F1 needed **both**.

Relational, NoSQL or Key-Value?

	Sharded MySQL (before)	NoSQL key-value (Bigtable)	Spanner
Data model	tables, a schema, SQL	a sorted map: (row, column, time) -> value	tables, a schema, SQL; every row **named by a primary key**
Scales out	only by sharding by hand	yes	yes, automatically
Transactions	ACID	one row only	ACID across rows, tables and continents
Replication	master-slave: failover risked data loss	eventually consistent across datacenters	synchronous, with Paxos

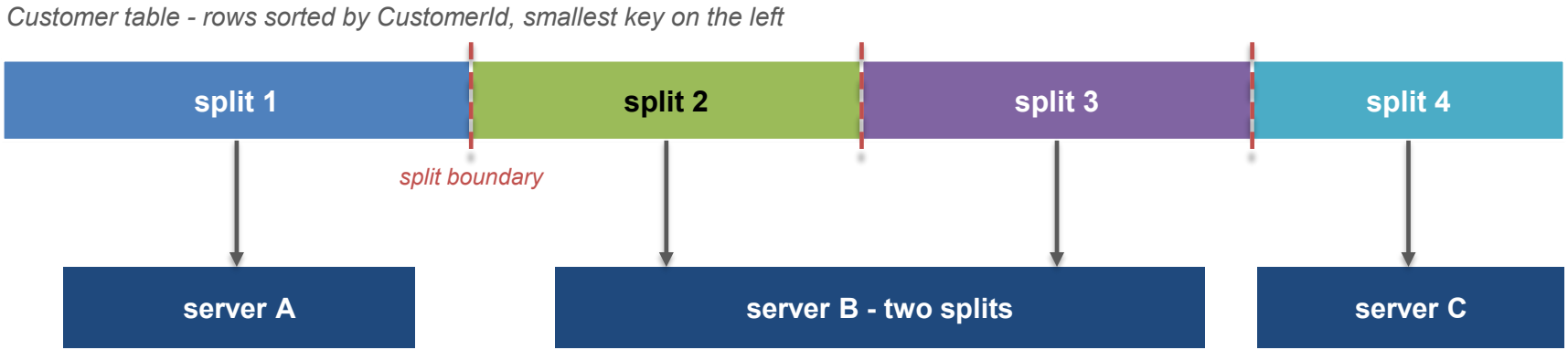
So which is it?

Relational on the outside, key-value underneath. Applications see tables, a schema and SQL. Inside, a table is a mapping from the **primary key** to the other columns - which is why every row must have a name.

Not NoSQL: it keeps what relational users expect and adds scale. [Corbett-2012] sections 1 and 2.3

Splits: What They Are For

A table with billions of rows cannot live on one machine. Spanner keeps the rows **sorted by primary key** and cuts the sorted table into **splits**.



Scale

A table can grow **beyond any one machine**.

Balance

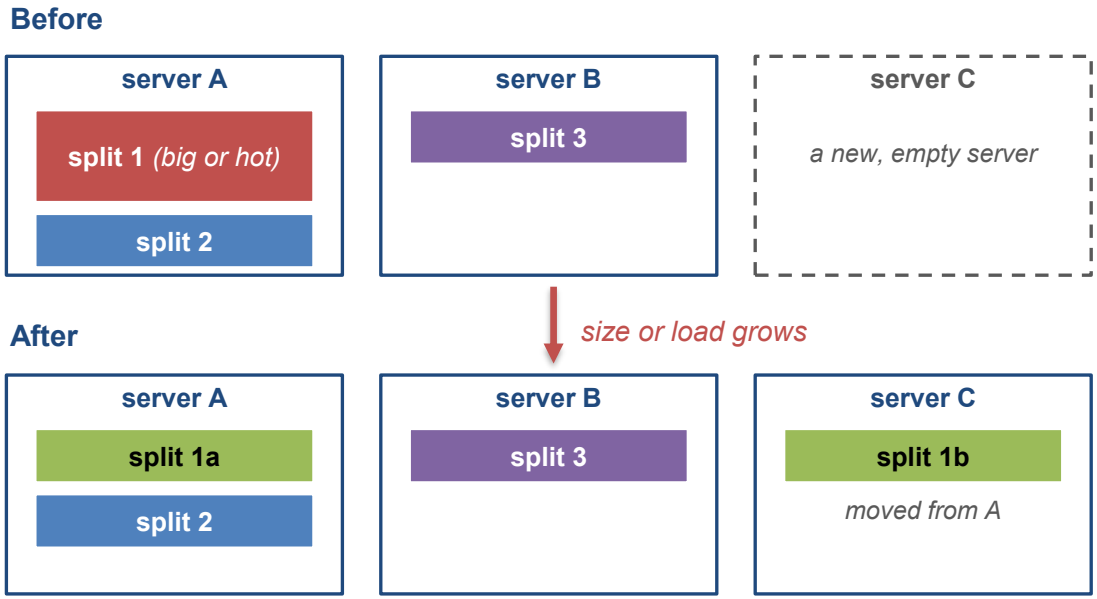
A busy split can **move** to a quieter server.

Replicate

Each split is **copied** to several datacenters.

A split is a range of contiguous rows. Illustrative: the boundaries and server count are invented.

Splits Change by Themselves



What triggers it

Spanner moves **boundaries** by itself, by **size** and by **load**.

What it can move

Splits move independently, even across datacenters, **online**.

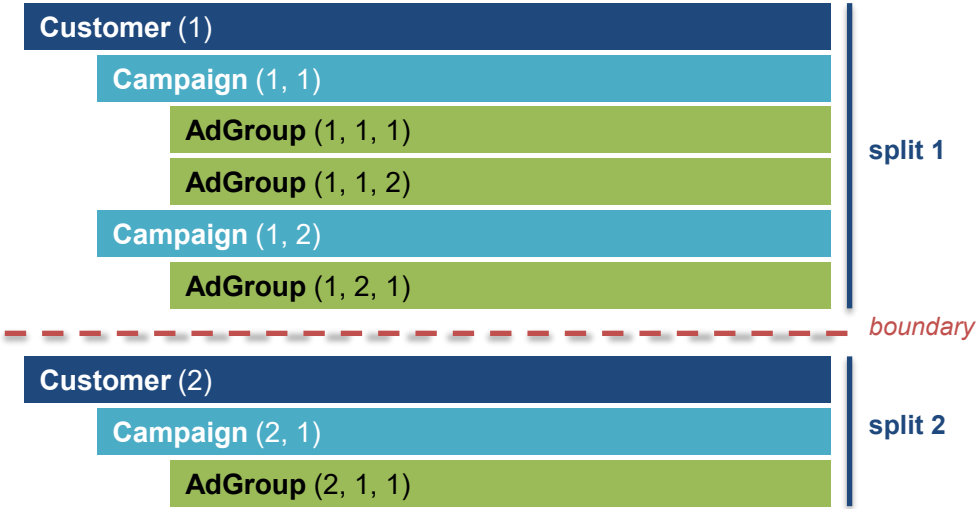
What you do

Nothing. F1's last MySQL resharding took **over two years**.

The catch: splits are key ranges, so a key that only grows (a timestamp, a counter) sends every new row to the *last* split. The key you choose is a performance decision.

Keeping Related Rows Together

The F1 schema: rows in stored order



Declared in the schema with `INTERLEAVE IN PARENT`: a child's primary key **starts with its parent's key**.

One advertiser, one split

Child rows are stored **under their parent**, so a boundary falls **between customers**, not through one.

One split, one group, no 2PC

A transaction on **one advertiser** usually stays in one Paxos group and **avoids 2PC**, which *"usually doubles"* commit latency.

Without it

Top-level tables carry **no locality guarantee**: related rows can land in different splits.

Context

- That is the problem. And we **already have the pieces** of the answer from the last three lectures - the mechanisms for **fault tolerance**, **transactional semantics** and **scalability**:
  - **Lecture 6** - **Paxos**, consensus: replicas agree on one order and survive failures.
  - **Lecture 7** - 2PL for isolation, WAL for atomicity and durability, on one node.
  - **Lecture 8** - **2PC**, atomic commitment across shards. And its limit: it *blocks* - unless the coordinator is made fault-tolerant with Paxos.
- **Spanner puts all three in one system** - and adds one thing the course has not needed yet: a **global notion of time**.

Today is less about new mechanisms than about what happens when you compose the ones you have.

Overview

A **sharded, replicated, transactional** database. The unit of sharding is a **split**.

Data is range-partitioned into splits

Each **split** holds a contiguous key range - the picture from the last few slides. There are a **dynamic number** of them: boundaries move with load and size.

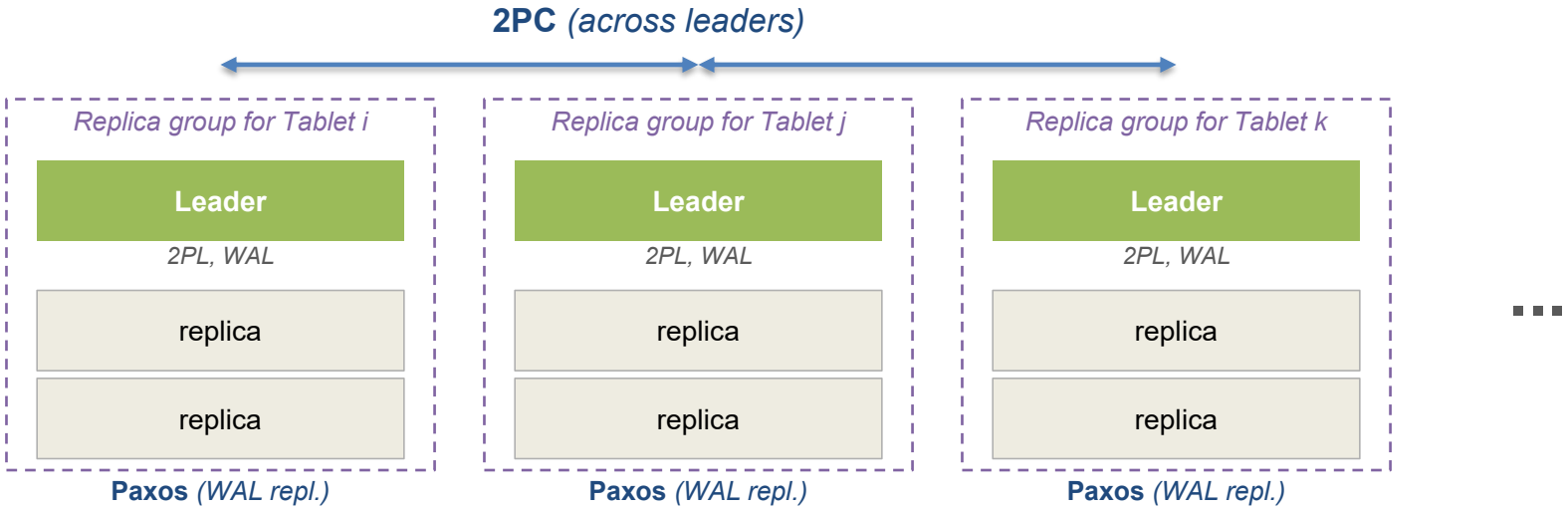
Each group of splits is its own replica group

Its replicas are kept in step by **Paxos** (Lecture 6), with one replica as **leader**. The 2012 paper calls what a group replicates a **tablet** - the word the figures use.

Transactions may span groups

So they need **2PC** (Lecture 8) across the group leaders - and the replica groups are **geo-distributed**, which is where the performance story starts.

Architecture



A transaction spanning tablets

`Begin(); update row1; update row2; update row3; Commit()` - if those rows live on different tablets, the tablet leaders run **2PC**, and one of them is the **TC**.

The Architecture is This Course

Every layer of that figure is a mechanism you already know.

Where	Mechanism	From	What it gives
Inside one replica	**2PL + WAL**	Lecture 7	isolation, atomicity, durability on one node
Down a replica group	**Paxos**	Lecture 6	the tablet's WAL is replicated by consensus
Across tablet leaders	**2PC**	Lecture 8	atomic commitment for multi-tablet transactions
And the new part	**TrueTime**	today	bounded physical time, used for *correctness*

The one new idea

Lecture 6 was careful: physical time is for **optimization**, never correctness. **Spanner overturns that** - it makes *bounded* physical time a correctness primitive.

Properties

- Two questions to ask of any system like this:
  - **Semantics?** What does it guarantee about what transactions see?
  - **Performance?** What does that guarantee cost?
- Spanner's answer to the first is unusually strong - and the answer to the second is what the rest of the lecture is about.

Semantics and Performance

Semantics

Spanner provides **atomic, externally consistent** transactions, at scale and with **geographic fault tolerance**.
(On the exact guarantee, see slide 22 - the terminology is worth getting right.)

Performance

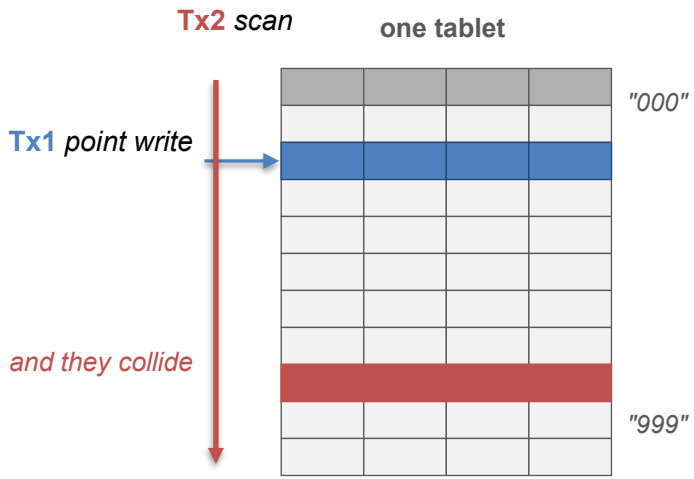
2PC, especially across geo-replicated groups, is expensive. Every multi-tablet commit crosses continents, and locks are held for the whole round.

And the authors' position, in their own words

"[S]ome authors have claimed that two-phase commit is too expensive to support, because of the performance or availability problems that it brings. We believe it is better to have application programmers deal with performance problems due to overuse of transactions as bottlenecks arise, rather than always coding around the lack of transactions."

More on Performance

What does 2PL + 2PC + Paxos across geographies actually cost?



*The cost is not any single operation - it is the **interaction between** transactions.*

Locks, on every row touched

A r/w transaction takes **r/w locks on every row it reads or writes**, and holds them until it commits.

Commit means 2PC over Paxos

So the hold time is a **geographic** round trip, not a local one.

Which is why scans hurt

A scan touches many tablet servers and blocks **every writer** to all of them.

What Can We Do?

Option 1: take fewer locks, or hold them for less time

This **weakens the semantics** - the result is no longer equivalent to a serial execution. And the authors have already told us what they think of that trade.

Option 2: treat two kinds of query differently

Split transactions into **point r/w queries** and **read-only scans**:

- **2PL** for the point queries, as before.
- **Lockless concurrency control** for the read-only scans.

Why option 2 is the good one

It **preserves the semantics** and removes the interference between scans and writes. Spanner takes this option - and it is what forces the rest of the lecture, because lockless reads need a **global clock**.

Lockless Concurrency Control

Multi-version concurrency control (MVCC)

Do not overwrite a row - **fork a new version** of it, copy-on-write style. Old versions remain readable.

IF you had a global notion of time...

...you could **timestamp every version**. A read-only scan then picks a timestamp *t* and reads, for each row, the latest version with timestamp <= *t*.

And that scan needs no locks at all

It sees a **consistent snapshot** as of *t*, cannot block any writer, and cannot be blocked by one. Reads and writes stop interfering.

So the whole design rests on: *"IF you had a global notion of time."* Do you? Lecture 6 said no.

What Global Time?

Lecture 6 said you could not trust physical clocks

What Must the Guarantee Be?

The scenario

Transaction **Ta** writes a. Later, **Tb** writes b. Later still, **Tc** reads both. What may Tc see?

The only acceptable answer

If Ta finished before Tb started, and Tb finished before Tc started, then Tc **must** see both writes: *post_a* **and** *post_b*. Anything else exposes an order that never happened.

The property has a name: EXTERNAL CONSISTENCY

"if one transaction completes before another transaction starts to commit", no client can ever see the second without the first. Google's phrasing: Spanner is *"semantically indistinguishable from a single-machine database."*

The next slide is about that name - the source deck called this linearizability, and the story is more interesting than a typo.

Which Guarantee, Exactly?

Worth two minutes, because **Google's own answer changed** - and the reason it changed is instructive.

2012, the Spanner paper

*"external consistency (or **equivalently, linearizability**)"* - the paper treats them as the same property.

Today, Google's documentation

External consistency is *"a **stronger** property than linearizability"*, which is *"a **special case** ... where a transaction can only contain a single read or write operation on a single object."*

Why both are defensible - and a third framing

Linearizability is stated **per object**; external consistency covers **multi-object transactions**. For single-operation transactions they coincide - hence 2012's "equivalently".
Google's **2023** engineering post drops the word entirely and frames the guarantee against **strict serializability**, calling the two *"somewhat synonymous"*.

How About Physical Clocks?

- **Option 1: give up on physical time.** Order events with logical clocks only - which is what Lecture 6 did, and it cannot give us external consistency, because logical clocks say nothing about *real* time.
- **Option 2: make physical time work.**
  - Google chose this, with **TrueTime** - a strong-semantic time service.
  - With it, you can actually get **external consistency** and **serializable isolation** at the same time.
- The rest of the lecture is what Option 2 costs, and how it is built.

Note what is being overturned: Lecture 6 used physical time only for leases, and only as an optimization.

TrueTime

- A **highly available, distributed clock** provided to applications on all Google servers.
- It lets applications generate **monotonically increasing timestamps across all servers** - not just within one machine.
- **The guarantee:** an application can compute a timestamp *T* guaranteed to be greater than any timestamp *T′*, **if *T′* finished being generated before *T* started** being generated.
- And this holds **even when *T* and *T′* are computed on different machines**. That last clause is the whole point.

Compare Lecture 6: logical clocks order events within a causal chain. TrueTime orders them in real time, across machines.

TrueTime Architecture

Reference clocks, per datacentre

Each datacentre has a set of **expensive but very accurate** clocks - **GPS receivers and atomic clocks** - used as *reference clocks*. Two independent failure modes, deliberately.

A time daemon, per machine

Every machine has a cheap, inexact **quartz** clock and runs a *time daemon* that polls **multiple reference clocks in multiple datacentres** and runs a synchronization protocol against them. The protocol **bounds the error**, to a few milliseconds under reasonable assumptions.

Between polls, the bound is widened on purpose

The daemon **applies a drift rate of 200 µsec/second** between synchronizations - far larger than the quartz clocks actually drift. The uncertainty is therefore *conservative by construction*.

The 200 µsec/s figure is load-bearing: the paper notes the worst problem would be a local clock drifting faster than that.

TrueTime Interface


```
typedef struct TT_interval {
    struct timeval earliest;
    struct timeval latest;
} TT_interval;

TT_interval TT_now(TT_interval tt);
```



epsilon

ε = latest − earliest, the *instantaneous uncertainty bound*. TrueTime never returns a point - it returns an **interval guaranteed to contain the true time**.

How big is ε? As reported in the 2012 paper

ε *"is typically a sawtooth function of time, varying from about **1 to 7 ms** over each poll interval"* - mean **≈4 ms**. The poll interval is **30 s**; the applied drift of 200 µsec/s accounts for **0–6 ms** of it; *"the remaining 1 ms comes from the communication delay to the time masters."*

[Corbett-2012] §3. Modern figures are much smaller - see slide 41.

Semantics and Use of TrueTime

- Call `TT_now()` and you get an interval *(earliest, latest)*. Then **wait until your own `TT_now().earliest` exceeds the `latest` you first got**.
- **The guarantee:** any *other* machine calling `TT_now()` after that point in real time will also see **its** `earliest` greater than your original `latest`.
- So we can build a **monotonically increasing timestamp across all machines**:
  - call `TT_now()`, take **latest** as the timestamp, then wait until `TT_now().earliest >` that value;
  - use the new `latest` as the next timestamp, and so on.
- **That wait is the price**, and it has a name: **commit wait**.

Everything Spanner does with time is this one pattern: pick `latest`, then wait out the uncertainty.

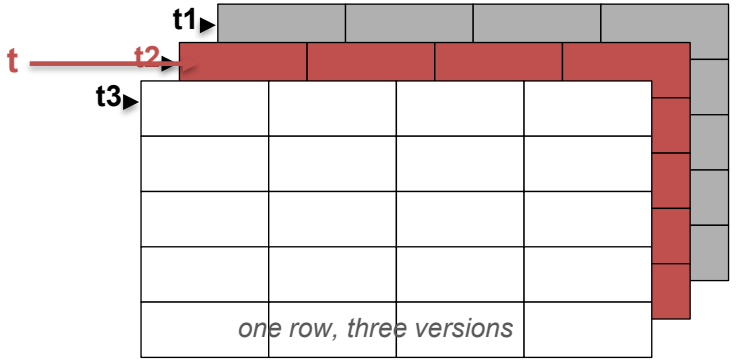
Serializable Transactions in Spanner

Read/write transactions

Executed with **2PL (+2PC +Paxos)** and **fork a new version** of each modified row, copy-on-write style. Every row a r/w transaction modifies is **tagged with that transaction's TrueTime timestamp**.

Read-only scan transactions

Executed **without locking**, but still isolated: they read a **consistent version** of each row, as of the scan's own TrueTime timestamp.



Which leaves two questions

(i) What timestamp should a transaction get? **(ii) When** should it be selected?

(i) What Timestamp to Select?

For a r/o transaction Tr to be isolated from a r/w Tw

If Tr reads any effect of Tw, then Tr's timestamp must be **> Tw's**.
If Tr does not read some effect of Tw, then Tr's timestamp must be **< Tw's**.

The key idea, in three lines

```
ts = TT_now().latest;
waitUntil(TT_now().earliest > ts);
// then run the transaction using ts as its timestamp
```

Why the wait is necessary

Without it, another machine could later pick a **smaller** timestamp for a transaction that really happened **later** - and the read order would contradict real time. The wait is what makes the timestamp order match reality.

Serializability among r/w transactions is already handled by 2PL. This is about the lockless readers.

(ii) When to Select a Timestamp?

Read-only transaction



Read/write transaction



Where the timestamp goes

The r/w transaction picks **tw after acquiring all its locks**, then **waits out ε** before releasing the first lock. So no reader can see its effects until its timestamp is safely in the past.

With Multiple Shard Servers

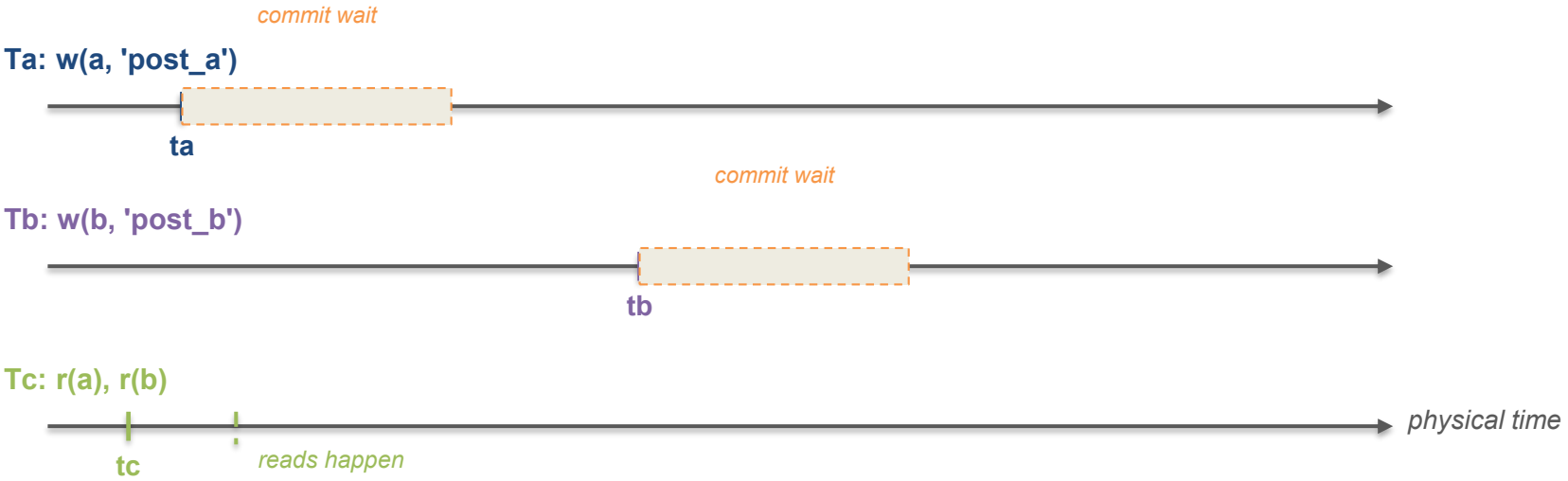
- With one tablet the story is simple: its **leader** picks the timestamp.
- Across tablets, the 2PC **coordinator** picks a timestamp that is **>= every participant's** prepare timestamp, and >= its own `TT_now().latest`.
- Then it does the **commit wait** before telling anyone the outcome.
- So the composition is: **2PL** for isolation, **Paxos** for durability of each tablet's log, **2PC** for atomicity across tablets, and **TrueTime** for a timestamp everyone agrees on.

All four mechanisms of the course, doing four different jobs in one commit path.

A Worked Example

Two writes and a read, four ways

Case 1: tc < ta < tb

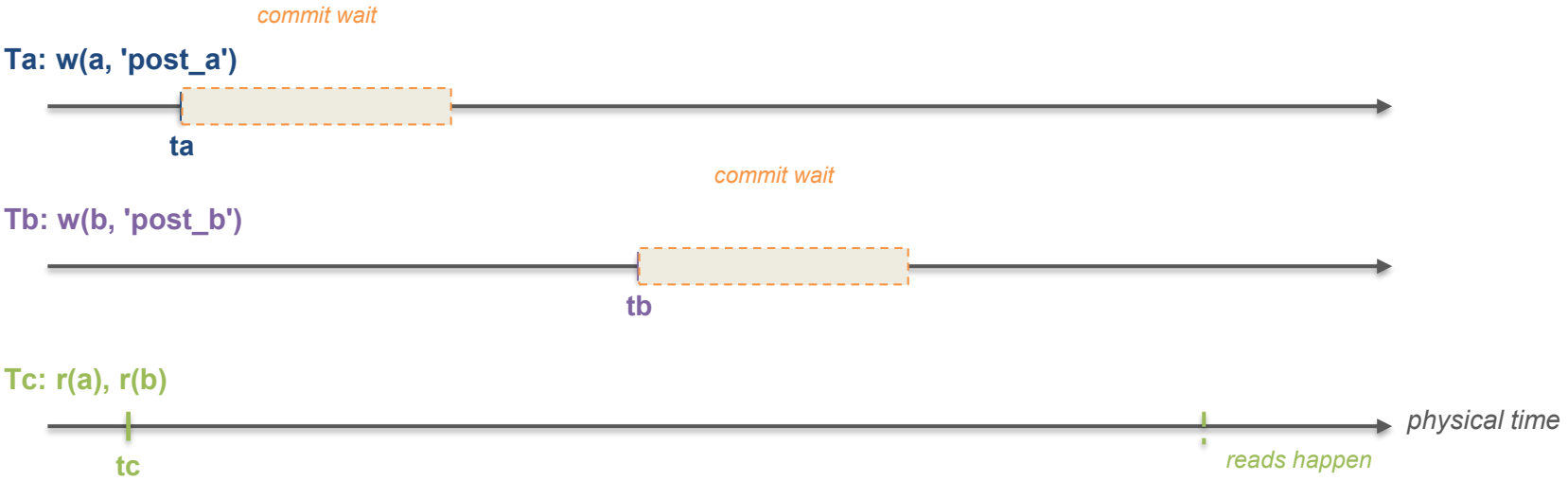


The read's timestamp precedes both writes.

What do the reads return?

pre_post_a and **pre_post_b** - neither write is visible. Tc picked its timestamp before either write committed, so it reads the older versions of both rows. Consistent, and no locks were taken.

Case 1': same, but Tc reads later

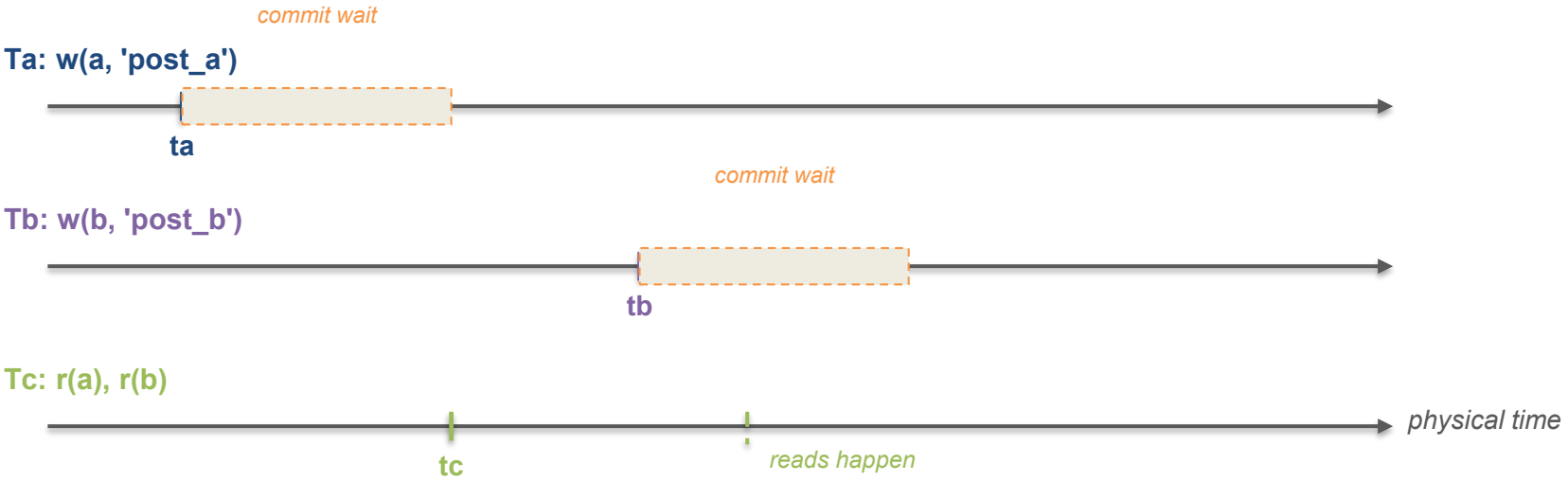


Same timestamps - but Tc physically performs its reads after both writes have landed.

What do the reads return?

Still pre_post_a and pre_post_b. This is the point of MVCC: what a read returns depends on its **timestamp**, not on when it happens to execute. The old versions are still there to be read.

Case 2: ta < tc < tb

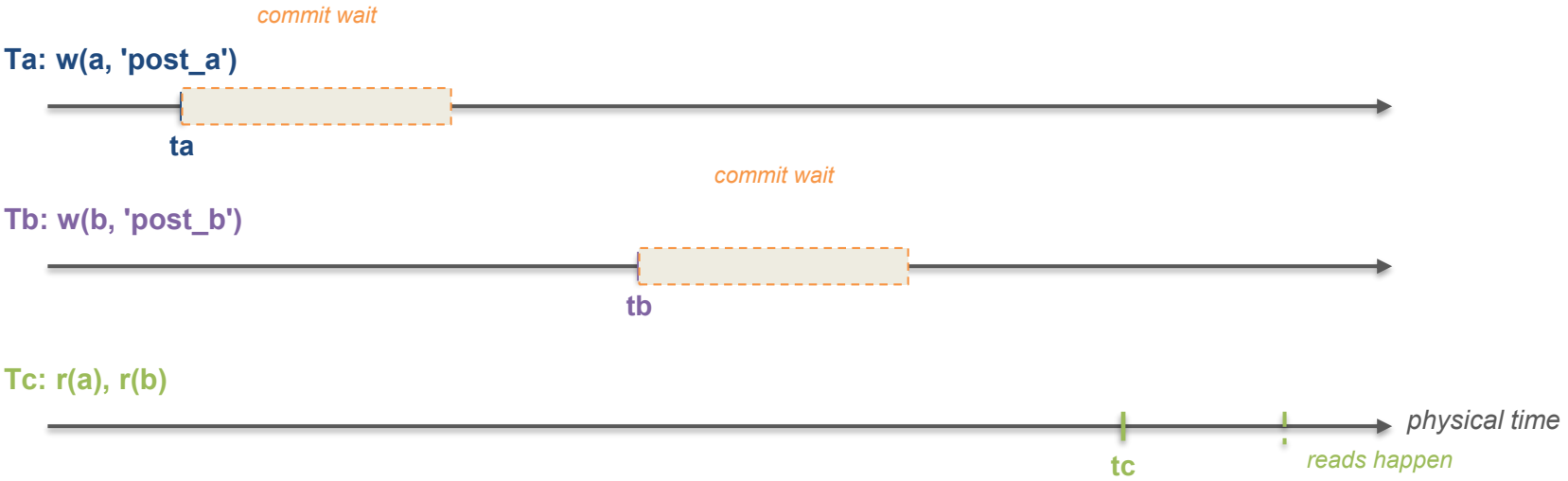


The read's timestamp falls between the two writes.

What do the reads return?

post_a and **pre_post_b**. Tc sees the first write and not the second - which is a correct serial order, because Ta really did finish before Tc started, and Tb had not.

Case 3: ta < tb < tc



The read's timestamp follows both writes.

What do the reads return?

post_a and **post_b**. And crucially this is the *only* possibility when Tc starts after Tb commits - which is what external consistency demands.

Implications of Commit Wait

The bigger the uncertainty, the longer the wait

Commit wait is **proportional to ε**. A wider TrueTime interval means a longer wait before locks can be released.

And locks are held during that wait

So commit wait **slows down dependent transactions** - anything wanting those rows queues behind it.

Which gives the headline

As time gets less certain, Spanner gets slower. Safety comes from the *correctness* of the (earliest, latest) interval; performance comes from its *size*.

That is the 2012 conclusion. It is still the right mechanism to understand - but the numbers have moved, see slide 41.

Spanner in 2026

What the numbers look like now, and what everyone else did instead

Spanner Became a SQL System

The system in the 2012 paper is not the product you can rent today.

The transition is documented

Bacon et al., *Spanner: Becoming a SQL System*, SIGMOD 2017 - the key-value store of the OSDI paper grew a full relational query engine.

Two SQL dialects today

GoogleSQL - shared with F1, Dremel and BigQuery (the 2017 paper calls it *"Standard SQL"*) - and a **PostgreSQL interface** that did not exist in either paper.

But the mechanism you just learned is unchanged

The 2017 paper, §1: *"strict two-phase locking ensures serializability within a given Paxos group, while two-phase commits (where different Paxos groups are participants) ensure serializability across the database."* **Still 2PL + Paxos + 2PC + TrueTime.**

How Big is Epsilon Now?

Then: 2012

ε *"varying from about **1 to 7 ms**"*, mean ≈4 ms - against a Paxos round trip of about **9 ms** and a measured commit wait of about **5 ms**. The wait was a real, visible cost.

Now: Google's 2023 engineering note

*"TrueTime provides Spanner servers with **less than 1 millisecond** clock uncertainty in the 99th percentile"* - and *"the vast majority of the time, Spanner's commit wait step **involves no waiting**."*

Why it vanished: the wait overlaps useful work

The timestamp is picked at step 1, the wait happens at step 3 - so **commit wait overlaps the Paxos round trip**, and is free while ε stays below it.

*So slide 38's mechanism is right and its warning is dated: clock uncertainty is a **latency floor you engineer below**.*

TrueTime is Now Commodity

In 2012 this needed a custom deployment. In 2026 it is a cloud API you can read from a file.

The same architecture, from AWS

The **Amazon Time Sync Service** uses *"a fleet of satellite-connected and atomic reference clocks in each AWS Region"* - GPS plus atomic, exactly Spanner's design - at *"microsecond-level precision"*.

And the same interface: ClockBound

It returns *"a current time estimate, an **upper error bound**, and a **lower error bound**"* - that is `TT_now()`. The hardware bound is readable per-instance **in nanoseconds** from `sysfs`.

Why this matters for the lecture

The mechanism you just learned is no longer exotic. Bounded physical time as a correctness primitive is **available to anyone** - which makes this design *more* relevant than in 2012, not less.

Five Other Bets on Time

Nobody else made Spanner's bet - and each pays elsewhere.

System	Time source	What it does about ε	Price
Spanner	GPS + atomic	**waits it out** (commit wait)	write latency *(now ≈0)*
CockroachDB	commodity NTP	**retries reads**	serializability only
YugabyteDB	HLC over the OS clock	**absorbs it** in a counter	conflicts rise with skew
TiDB	central timestamp oracle	**eliminates** disagreement	a coordination round trip
Calvin	none - deterministic	**removes physical time**	no external consistency
Aurora DSQL	atomic/GPS **+ HLC**	**tight bounds *and* HLC**	OCC aborts

CockroachDB's own summary: *"While Spanner always waits after writes, CockroachDB sometimes retries reads."*

Chubby

The lock service Paxos is for

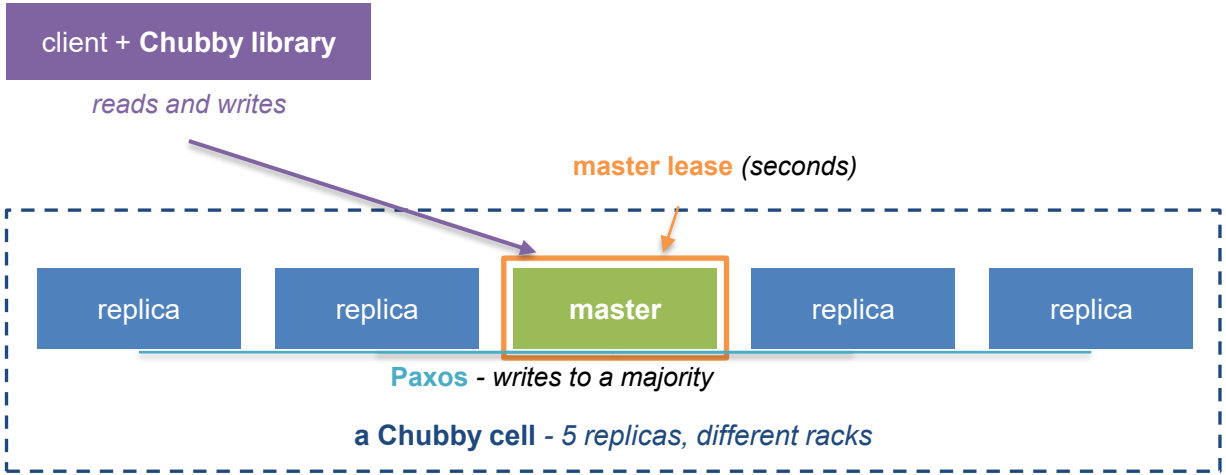
Why Chubby, and Why Now?

- Spanner's architecture has **Paxos groups with leaders**. Something has to *elect* those leaders, and it must not ever elect two.
- **That something is Chubby** - a fault-tolerant lock service, and the most direct payoff of lecture 6 in the whole course.
- Its own summary is the thesis: *"Chubby uses Paxos for everything"* - propagating writes, choosing a master, even adding new Chubby servers to a cell.
- **And it is honest about what it is.** From the paper: *"Building Chubby was an engineering effort ... it was not research. We claim no new algorithms or techniques."*
- Which is exactly why it is worth studying: it is what happens when a protocol from lecture 6 has to survive contact with real clients.

What Chubby Is

- A **lock service** for a loosely-coupled distributed system - not a storage system, though it stores a little.
- The client interface is deliberately familiar:
  - **Whole-file advisory locks** - think `.lck` files.
  - **Notification of events** - file modified, master failed over, child added. Think `inotify`.
- Primary goals, in the paper's own order: **reliability, availability, and easy-to-understand semantics.** *Note what is absent: throughput.*
- Used inside Google by **GFS and BigTable**, to elect masters and to store small amounts of metadata - typically the root of a distributed data structure.

A Chubby Cell



Writes go to the master, which propagates them by Paxos and replies once a **majority** has them. **Reads** the master satisfies **alone** - which is why the lease matters so much.

Two Masters Would Be a Disaster

The invariant: at any instant, at most one master.

The scenario

A is master. It gets partitioned from **B**.

B acts

B times out talking to A, concludes A is dead, and proposes itself as master. The others agree.

But A never heard

A keeps acting as master, answering reads from its own copy - which is now **stale**.

The fix: Paxos plus a lease

A master change is accepted **only once the old master's lease has expired**. A master knows it holds the role for at least the lease period.

The Interface: A Tiny File System

- Chubby exposes a **strict tree of files and directories** - no symbolic links, no hard links.
- `/ls/foo/wombat/pouch`
  - `ls` - *lock service*, common to every name.
  - `foo` - the **Chubby cell**, used in a DNS lookup to find its master.
  - the rest - the name **inside** the cell.
- **The right mental model:** these files are *locks with a little bit of content* - typically the identity and location of a primary.
- Normal operations are supported (create, delete, open, write), plus a **reader/writer lock** on any node.

Events and the API

Clients subscribe to events

File contents modified - the standard way to track a service's location.
Master failed over.
Child node added / removed / modified.
Handle became invalid - probably a communication problem.
Lock acquired, and locks conflicting - both rarely used.

The API

`Open()` - *mode, events, lock-delay*
`Close()`
`GetContentsAndStat()`, `GetStat()`, `ReadDir()`
`SetContents()`, `SetACL()`, `Delete()`
Locks: `Acquire()`, `TryAcquire()`, `Release()`
Sequencers: `GetSequencer()`, `SetSequencer()`, `CheckSequencer()`

Why sequencers exist: the locks are *advisory*, so a client that has lost its lock without noticing can still issue a write. A **sequencer** is a token naming the lock and its generation; the client passes it along, and the receiving server checks it is still valid before acting.
This is the same problem, and the same shape of fix, as a fencing token in any lease-based system - and it is why advisory locks are safe to build on at all.

Primary Election in Ten Lines


```
Open("/ls/foo/OurServicePrimary", "write mode");
if (successful) {
    // I am the primary
    SetContents(primary_identity);
} else {
    // I am a replica
    Open("/ls/foo/OurServicePrimary", "read mode",
         "file-modification event");
    when notified of file modification:
        primary = GetContentsAndStat();
}
```

This is the whole point of the section.

Electing a primary - the problem lecture 6 needed Paxos to solve - is now **one `Open()` call**.

The Paxos is still there. It is just *inside* Chubby, written once, by one team.

And that is what a good abstraction buys. Every service at Google that needs a primary gets fault-tolerant election without implementing consensus - which is also why a bug in Chubby would have been a very bad day.

Five Nodes, and Bad Clients

- **Five replicas per cell turns out to be enough** - but only because Chubby's users are made to be careful.
- **The intended use is as a name service** - BigTable and anything needing a more consistent DNS.
- **The abuses were real:**
  - Someone built a **publish/subscribe system** on Chubby.
  - Repeated `open`/`close` calls to **poll** a file. *(The fix: cache file handles.)*
- **The general fix was social, not technical:** review clients' code before they are allowed to use a shared Chubby cell.
- *Which is a real engineering finding, and the reason the paper says it is about "what we did and why".*

Chubby: What to Take Away

- A **lock service** with a UNIX-like file interface, built for reliability and availability rather than throughput.
- **Paxos for everything** - propagating a write, choosing a master, and even adding a new server to a cell.
- **Physical time became load-bearing for safety.** The master lease is what stops a partitioned old master serving stale reads. Consensus alone could not do that.
- **The one-sentence summary, from the source:** Paxos *"transforms a multi-node service into something that looks very much like one fault-tolerant, albeit slower, server"*.
- Which is this course's core goal, achieved - and the same sentence you could write about Spanner, with *slower* replaced by *more expensive*.

BigTable

The storage model Spanner inherited

Why BigTable, and Why Last?

- Spanner's architecture slide is drawn in terms of **tablets**. BigTable is where that abstraction comes from.
- It is also **the system Spanner had to replace**, and the reason is a single sentence from its own slides:
  - *"Data row is the only unit of atomicity in BigTable"* - **no multi-row transactions**, no integrity constraints, no relational model.
- **Closing exactly that gap is what Spanner is for.** Read in this order, Spanner stops being an exotic system and becomes the obvious next step.
- Chang, Dean, Ghemawat, Hsieh, Wallach, Burrows, Chandra, Fikes, Gruber - **OSDI 2006**, the same conference as Chubby.

What BigTable Is For

- *"A distributed storage system for (semi-)structured data."*
- **Scalable**, at the scale the name implies: thousands of servers, terabytes in memory, **petabytes on disk**, millions of reads and writes per second, with efficient scans.
- **Self-managing** - servers can be added or removed dynamically, and the system rebalances load itself.
- **And it was everywhere**: web indexing, personalized search, Google Earth, Google Analytics, Google Finance.
- Built on three things this course has now seen: **GFS** for raw storage, a **scheduler** for placement, and **Chubby** for master election and location bootstrapping.

The Data Model

"A BigTable is a sparse, distributed, persistent multi-dimensional sorted map."

(row: string, column: string, timestamp: int64) -> string

row key	contents:	anchor:cnnsi.com	anchor:stanford.edu	lang:
com.cnn.www	"<html>..."	"CNN"	"CNN homepage"	"EN"

column families are the unit of access control; qualifiers give unboundedly many columns

Row keys are arbitrary strings, sorted lexicographically - up to 64 KB, usually 10-100 bytes. Here they are **reversed URLs**, and that choice is the entire locality story.

Access to data in a row is atomic - and the row is the *only* unit of atomicity. No multi-row transactions, no integrity constraints, no relational model. *Remember this when we get to the summary.*

Row Keys Are a Locality Decision



Rows close together lexicographically are stored on **one or a few machines**. So the programmer chooses locality by choosing key names.

www.cnn.com (natural order)

```
www.cnn.com
money.cnn.com
edition.cnn.com
```

Three unrelated points in the key space - three different machines.

com.cnn.www (reversed)

```
com.cnn.edition
com.cnn.money
com.cnn.www
```

Adjacent. One machine answers a site-local query.

So which key gives better locality for site-local queries? Reversing the DNS name clusters a site's pages together, so matches from one site are retrieved by touching the **minimum number of machines**.
*With no joins and no secondary indexes, the row key **is** your index and its sort order **is** your physical layout - so naming rows is a performance decision.*

Columns and Timestamps



Columns: family:qualifier

Column family - the unit of **access control**, carries type information, and there are only a **few** of them.
Qualifier - unboundedly many columns per family, extremely sparsely populated. `anchor:cnnsi.com` is a family plus a qualifier.

Timestamps: many versions

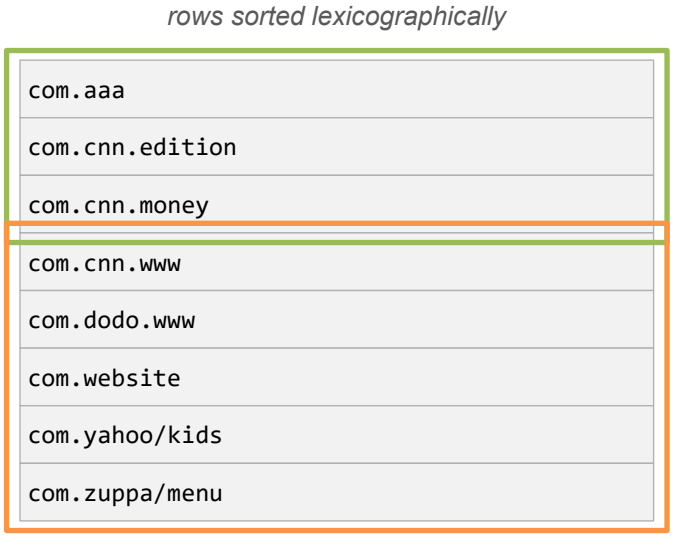
New writes default to now; clients may set them explicitly.
Read options: *"most recent N values"*, or *"all values in a timestamp range"*.
Families can be told to **keep only the last N versions**, or to discard anything older than **T seconds**.

You have seen this before, an hour ago. Multiple timestamped versions of a row, with reads selecting a version - that is **MVCC**, and it is exactly the mechanism Spanner's lock-free read-only scans use. BigTable had the versions; what it lacked was a *globally meaningful* timestamp to select them by.
So the gap TrueTime fills was never the storage - it was the clock. Everything else was already here, in 2006.

The API

- **Metadata:** create and delete tables and column families, change metadata.
- **Writes - single-row, atomic:** `Set()` writes cells in a row, `DeleteCells()`, `DeleteRow()`.
  - *And that is the whole write story. There is no multi-row transaction to be had.*
- **Reads - a scanner abstraction**, and each row read is atomic:
  - Restrict to a **row range**; ask for one row, or all rows.
  - Ask for all columns, certain **families**, or specific columns.
  - Ask for **certain timestamps only**.
- *The read side is expressive. The write side is deliberately minimal - and that asymmetry is the design.*

Tablets: How a Table Is Split



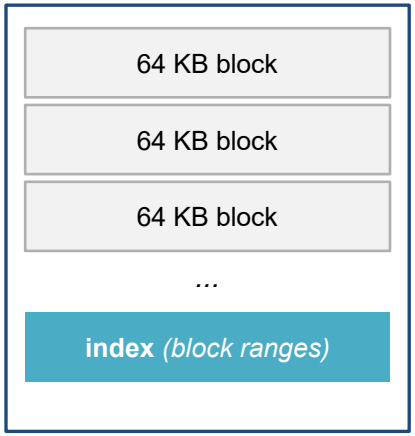
What a tablet is

A table is partitioned by **row-key range** into tablets of **100-200 MB**.
Each tablet is served by **exactly one** tablet server.
Tablets are stateless - all their state is in GFS - so a tablet server can restart, or be replaced, at any time.
Tablets **split** when they grow, and **merge** when they shrink.

This is the abstraction Spanner inherited - and then changed in one crucial way. A Spanner tablet is replicated by its **own Paxos group**, so the tablet itself is fault-tolerant. A BigTable tablet is served by one machine and relies on **GFS** underneath it for durability.

Inside a Tablet: SSTables

A tablet stores its rows as a set of **SSTable** files, and the SSTables live in **GFS**.



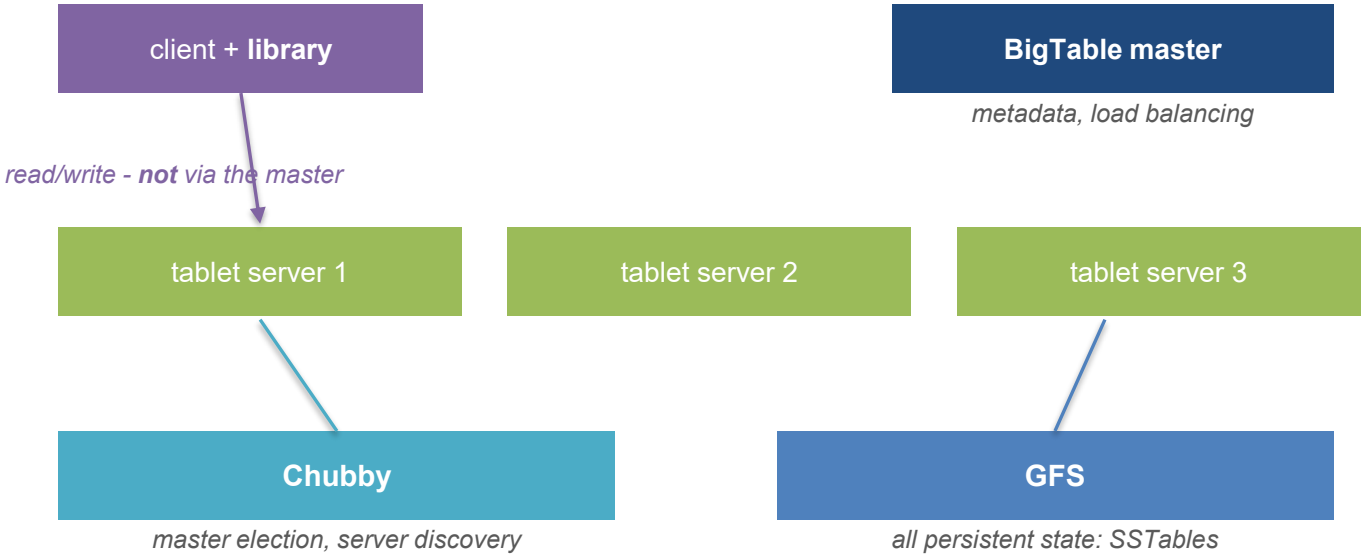
one SSTable

An SSTable is immutable

An **immutable, sorted** file of key-value pairs, keyed by **<row, column, timestamp>**.
Appends only - no in-place updates. A delete is itself an append.
So why refuse updates? Immutability means no locking on read, safe concurrent access, cheap caching - and it fits GFS, which is itself append-optimised.

Appending instead of overwriting should feel familiar. It is the write-ahead log from lecture 7 and the copy-on-write versioning Spanner uses, in a third guise. *In storage systems, the answer is very often "append, never overwrite".*

The Serving Stack



Everything above the two bottom boxes is stateless. The master does metadata and load balancing; tablet servers serve data; clients talk **directly** to tablet servers, never through the master.

Finding a Tablet: A Recursive Trick

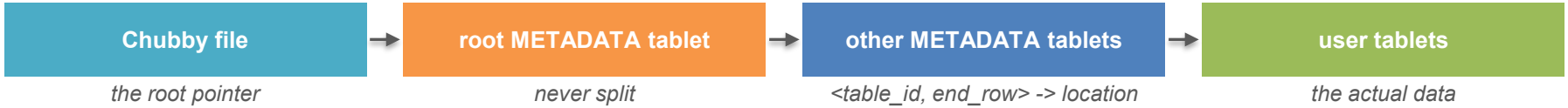
Tablets move between servers, so given a row, **how does a client find the machine serving it?**

Not via the master

A central server on every lookup path would be the bottleneck of the whole system.

Instead: store it in BigTable

Keep the tablet-location tables **inside the BigTable cell itself**. *Recursive.*



Three levels is enough, and the arithmetic says why: each METADATA record is about **1 KB** and a METADATA tablet caps at **128 MB**, which addresses roughly **2^34 tablets** - far more than anyone needed. *A B+-tree, built out of the system's own storage.*

Assignment via Chubby Locks

One tablet, one server - and the whole protocol runs on Chubby locks.

A tablet server starts up

It **acquires an exclusive lock** on a uniquely named file in a Chubby directory. The master watches that directory, and so discovers it.

A tablet server loses its lock

It **stops serving** at once, and retries while its file exists. If the file is gone, it **kills itself**.

The master suspects a server

The master tries to **take that server's lock itself**. Success means Chubby is alive and the server is not - so it **deletes the file** and reassigns.

The master polices itself

If its own Chubby session expires, the master kills itself.

BigTable: What to Take Away

- A scalable distributed store for semi-structured data, with a **multi-dimensional map** interface: `<row, column, timestamp> -> value`.
- **Atomic reads and writes within a row** - and, crucially, **nowhere else**.
- **Statelessness is the key design philosophy**, and it is what makes the system self-managing: every BigTable server, master included, is stateless. All state lives in **GFS** and **Chubby**.
- Which means BigTable's reliability is *borrowed*: it leverages **strong-semantic operations in the layers below** - atomic appends in GFS, exclusive file locks in Chubby.
- **And that is the honest limit of this design.** Borrowed guarantees get you atomic rows. They do not get you a transaction across two rows on two machines - for that, someone has to build Spanner.

Three Systems, One Stack

Spanner	*transactions* 2PL + WAL per tablet, Paxos per tablet, 2PC across tablets.	*External consistency - paid for with atomic clocks.*
BigTable	*storage model* Tablets over SSTables. Stateless servers, borrowed guarantees.	*Atomic rows, nothing more.*
GFS	*raw storage* Append-optimised, replicated chunks.	*Atomic appends, nothing more.*
Chubby	*coordination* Paxos + leases. Elects masters, holds tiny metadata.	*Time is load-bearing: the lease.*

"Read bottom-up": You can follow the dependency chain clearly from the ground up:
And every one of them pays for its guarantee: in a lease, in an append-only API, in single-row atomicity, or in hardware.

- **Chubby** anchors the base (providing coordination and master elections).
- **GFS** builds directly on top of Chubby.
- **BigTable** utilizes GFS for its distributed storage framework.
- **Spanner** crowns the stack, orchestrating globally distributed transactions over the underlying layers.

Next Up

- **That is the end of the mechanism arc.** Lectures 6 to 9 built one system from the ground up: consensus, locks and logs, commitment, time - and then three real systems that use them.
- **Next: broader semantics.** What if you do *not* need external consistency? What weaker guarantees exist, and what do they buy?
- BigTable is already a partial answer - it gave up multi-row atomicity for scale. The next lecture makes that trade systematic.
- **And a closing thought:** every mechanism in these four lectures traded something. 2PL trades concurrency, 2PC trades availability, Paxos trades a round trip, a lease trades a few seconds of availability, TrueTime trades hardware. *There is no free consistency.*

Key Papers

- **[Corbett-2012]** Corbett et al. (26 authors). *Spanner: Google's Globally-Distributed Database.* **OSDI 2012, pp. 251-264**. Extended as ACM **TOCS 31(3), art. 8**, 2013, DOI `10.1145/2491245` *(the journal title drops the hyphen)*. TrueTime is **section 3**; there is no separate TrueTime paper.
- **[Burrows-2006]** Burrows. *The Chubby Lock Service for Loosely-Coupled Distributed Systems.* **OSDI 2006** - the lock service, and the engineering consequences of deploying Paxos.
- **[Chang-2006]** Chang, Dean, Ghemawat, Hsieh, Wallach, Burrows, Chandra, Fikes, Gruber. *Bigtable: A Distributed Storage System for Structured Data.* **OSDI 2006** - tablets, SSTables, and single-row atomicity.
- **[Ghemawat-2003]** Ghemawat, Gobioff, Leung. *The Google File System.* **SOSP 2003** - the append-optimised storage both of the above sit on.
- **[Bacon-2017]** Bacon et al. (17 authors). *Spanner: Becoming a SQL System.* **SIGMOD 2017, pp. 331-343**, DOI `10.1145/3035918.3056103` - the current reference for the transaction mechanism.
- **[Lamport-1998]** *The Part-Time Parliament.* ACM **TOCS 16(2):133-169** - Paxos, which all three systems above run on.
- **[Gray-Lamport-2006]** *Consensus on Transaction Commit.* ACM **TODS 31(1):133-160** - 2PC over Paxos, which is what Spanner is.
- Also: Google Cloud Spanner *TrueTime and external consistency* docs; Google's 2023 *Spanner under the hood* engineering post; AWS *Amazon Time Sync Service* and *ClockBound* documentation.